Dhirubhai Ambani University
Technology
Formerly DA-IICT

PC-406: B.Tech. Mini Project-1

Graphify: Interactive Graph Algorithm Visualizer

Under the Guidance of

Prof. Rahul Muthu

Github: Repository Link

Project Live Demo: Click here

Submitted by

Atik Vohra (202301447)

Vivek Parmar (202301475)

Contents

1 Introduction

1.1 Motivation

Understanding graph algorithms is hard for many students. Most students can read code or formulas, but they cannot clearly see:

- Which node is visited now
- Which edge is selected now
- How the final answer is built step by step

Graph algorithms are dynamic in nature. While code and formulas explain the logic, it is hard to visualize what actually happens during execution. Students find it challenging to track which node is being processed, which edges are selected, and how the final result is formed step by step. Static diagrams in textbooks are not enough to show this behavior clearly.

1.2 Problem Statement

Graph algorithms become harder to understand as the size of the graph increases. With many nodes and edges, the structure looks complex and confusing. It is difficult to keep track of visited nodes, active nodes, and selected edges.

Most existing learning methods focus on theory or code, but they do not provide interaction. There is a lack of a simple platform where students can test multiple algorithms and see how they work visually.

1.3 Objective

To solve this problem, Graphify is developed as an interactive web application. It allows users to:

- Input graphs using simple adjacency list format
- Run different algorithms step by step
- See clear visual representation with color coding
- Understand how algorithms work in practice
- Compare different algorithm behaviors

This makes learning more intuitive and helps bridge the gap between theory and practice.

2 Technology Used

2.1 React 19 (Frontend Framework)

What it is: React is a JavaScript library for building user interfaces using components.

Why we use it: It allows dynamic updates of the graph visualization when algorithms execute step-by-step.

What it provides:

- Component-based architecture
- Automatic UI updates
- Fast rendering with Virtual DOM

2.2 React Router DOM (Routing)

What it is: A library for handling navigation in React applications.

Why we use it: It enables smooth navigation between different algorithm pages without reloading the application.

What it provides:

- Client-side routing
- URL-based navigation

2.3 Vite (Build Tool)

What it is: A modern frontend build tool and development server.

Why we use it: It provides very fast startup and instant updates during development.

What it provides:

- Fast development server
- Transpiles JSX into JavaScript using esbuild and create React Elements
- Optimized production builds

2.4 Tailwind CSS 4 (Styling)

What it is: A utility-first CSS framework.

Why we use it: It allows quick and consistent UI design using predefined classes.

What it provides:

- Utility-based styling
- Responsive design support

2.5 Additional Libraries

- **Lucide React**: Icon library for UI elements
- **Shadcn UI**: Prebuilt Components
- **ESLint**: Code quality and style checking

3 System Design

The Graphify system is designed as a client-side interactive web application. All major functionalities such as input processing, algorithm execution, and visualization are handled within the frontend using React.

3.1 System Architecture

The system consists of multiple layers that work together to provide an interactive learning experience.

- **User Interface Layer:** Allows users to input graphs, select algorithms, and control execution using buttons such as Next Step and Reset.
- **Input Processing Layer:** Parses the adjacency list input and converts it into internal data structures such as nodes, edges, and adjacency lists.
- **Algorithm Engine:** Executes graph algorithms step-by-step. Each algorithm maintains its own state such as visited nodes, distances, or flow values.
- **State Management:** Keeps track of the current execution state including active nodes, visited nodes, and selected edges.
- **Visualization Layer:** Converts algorithm state into visual representation using color coding and highlighting.
- **Rendering Layer:** Updates the user interface dynamically after each step to reflect the current state of execution.

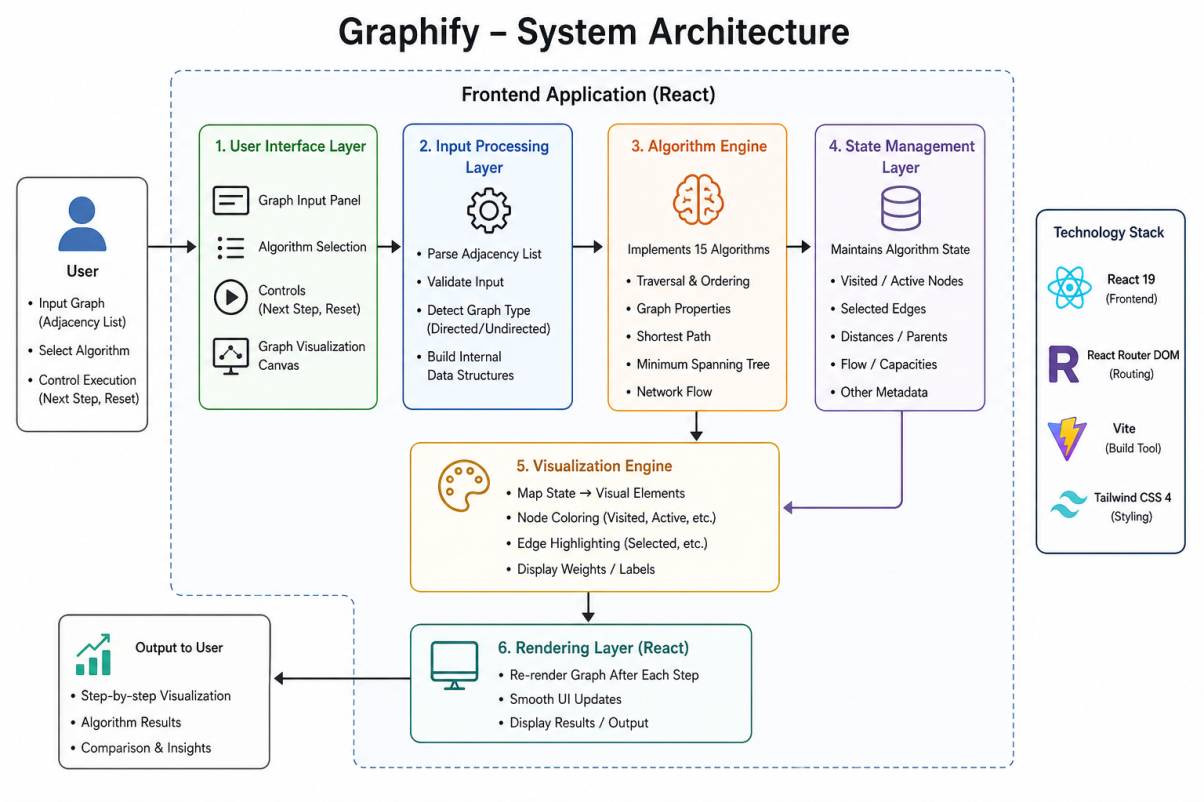

Figure 1: System Architecture of Graphify

3.2 System Workflow

Step 1: Input Graph

The user enters a graph in adjacency list format.

Example for an unweighted graph:

```
A: B C
B: A D
C: A D
D: B C
```

For weighted graphs:

```
A: B(5) C(3)
B: A(5) D(2)
C: A(3) D(4)
D: B(2) C(4)
```

Step 2: Parse and Normalize

The input text is parsed into an internal graph representation:

- Nodes array: list of all vertices

- Edges array: list of all connections
- Adjacency list: for efficient neighbor lookup
- Weight map: used for weighted algorithms

Step 3: Validate Graph

Basic validation checks are performed:

- No missing node references
- Correct input format
- Valid graph type for the selected algorithm (e.g., DAG for topological sort)
- Proper weight format (for weighted graphs)

Step 4: Initialize Algorithm State

Each algorithm initializes its required data structures:

- **BFS/DFS:** Queue/Stack and visited set
- **Dijkstra:** Priority queue, distance array, parent array
- **MST:** Union-Find and sorted edge list
- **Flow:** Residual graph and flow values

Step 5: Execute Step-by-Step

Each click of the "Next Step" button:

- Updates the internal algorithm state
- Maintains data structures for visited, unvisited, and active nodes/edges
- Generates a log message explaining the current step
- Marks currently active nodes and edges
- Updates intermediate results

Step 6: Visual Update

At each step, the state of nodes and edges (visited, unvisited, active, and selected) is stored internally. Based on this state, the graph visualization is updated dynamically. The colors of nodes and edges change accordingly to reflect the current execution of the algorithm.

Step 7: Final Result

When the algorithm completes:

- The final path, tree, or flow value is displayed
- Statistics such as total cost or path length are shown
- The user can reset and try different inputs

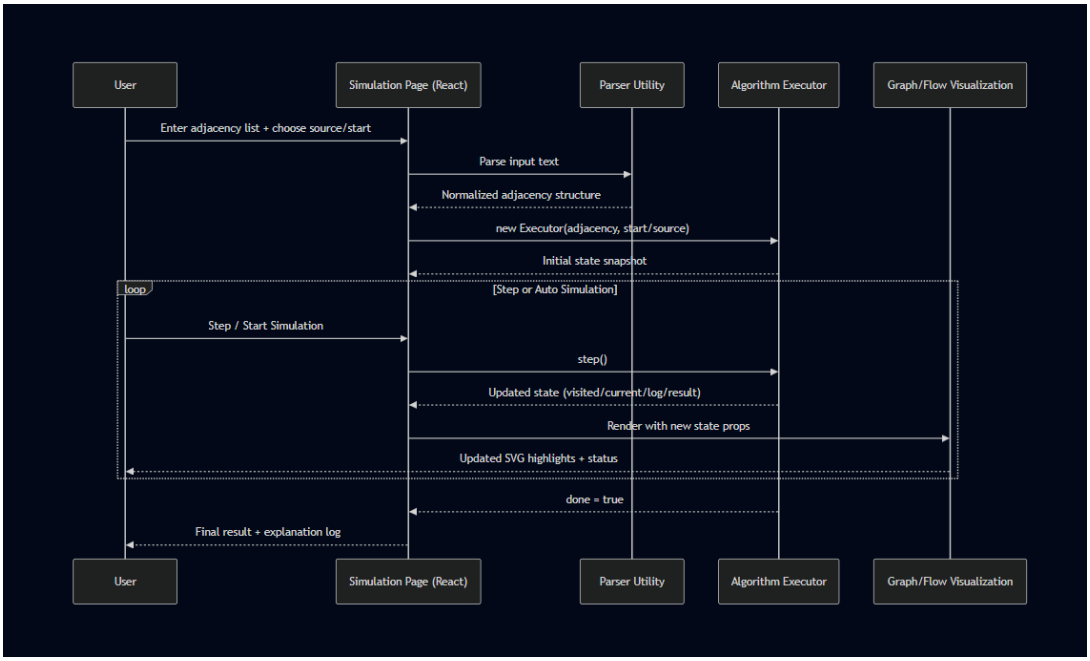

Figure 2: User Interaction Sequence Diagram

4 Implemented Algorithms

This chapter provides detailed explanations of all 15 algorithms implemented in Graphify.

4.1 Traversal and Ordering Algorithms

4.1.1 Breadth-First Search (BFS)

BFS explores a graph level by level, starting from a source node. It visits all neighbors of the starting node first, then all neighbors of those neighbors, and so on.

Use Cases:

- Finding shortest path in unweighted graphs
- Level-order traversal of trees
- Finding all connected nodes within a certain distance
- Web crawling

How It Works:

1. Start with a source node in a queue
2. While queue is not empty:
    - Remove front node from queue
    - Mark it as visited
    - Add all unvisited neighbors to queue

Time Complexity: O(V + E) where V is vertices and E is edges

4.1.2 Depth-First Search (DFS)

DFS explores a graph by going as deep as possible along each branch before backtracking. It follows one path until it hits a dead end, then goes back and tries another path.

Use Cases:

- Detecting cycles in graphs
- Topological sorting
- Finding connected components
- Maze solving
- Path finding

How It Works:

1. Start with a source node on a stack (or use recursion)
2. While stack is not empty:
    - Pop top node from stack
    - Mark it as visited
    - Push all unvisited neighbors to stack

Time Complexity: O(V + E)

4.1.3 Topological Sort

Topological sort orders nodes in a directed graph so that for every edge from node A to node B, A appears before B in the ordering. This only works on graphs with no cycles (DAGs - Directed Acyclic Graphs).

Use Cases:

- Task scheduling with dependencies
- Build systems (compile files in correct order)

- Course prerequisite planning
- Dependency resolution

How It Works (Kahn's Algorithm):

1. Calculate in-degree (incoming edges) for all nodes
2. Add all nodes with in-degree 0 to queue
3. While queue is not empty:
    - Remove front node, add to result
    - Decrease in-degree of all neighbors
    - If a neighbor's in-degree becomes 0, add it to queue

Time Complexity: O(V + E)

4.2 Graph Property Algorithms

4.2.1 Bipartite Check

Checks if a graph can be divided into two groups where no two nodes in the same group are connected. This means you can color the graph with two colors such that no adjacent nodes have the same color.

Use Cases:

- Matching problems (jobs to workers, students to projects)
- Scheduling conflicts

How It Works:

1. Use BFS/DFS to traverse the graph
2. Try to color each node with opposite color of its parent
3. If a node is already colored and neighbor has same color, graph is not bipartite

Time Complexity: O(V + E)

4.2.2 Connected Components

Finds all separate groups (components) in a graph where nodes within each group can reach each other, but nodes in different groups cannot.

Use Cases:

- Finding isolated clusters in social networks

- Identifying separate road networks

How It Works:

1. Start with all nodes unmarked
2. For each unmarked node:
    - Run BFS/DFS to find all reachable nodes
    - Mark them as one component
3. Count how many times you started BFS/DFS

Time Complexity: O(V + E)

4.2.3 Cycle Detection

Determines if there is a path in the graph that starts and ends at the same node, forming a loop.

Use Cases:

- Detecting deadlocks in systems
- Checking for circular dependencies
- Validating DAGs (for topological sort)

How It Works (for undirected graphs):

1. Use DFS to traverse the graph
2. Keep track of parent nodes
3. If you visit a node that's already visited and it's not your parent, a cycle exists

Time Complexity: O(V + E)

4.2.4 Whitney Connectivity Theorem

Whitney's theorem states that for any graph G:

$$\kappa(G) \leq \lambda(G) \leq \delta(G)$$

where $\kappa(G)$ is vertex connectivity, $\lambda(G)$ is edge connectivity, and $\delta(G)$ is the minimum degree of the graph.

How it works:

We computed vertex connectivity $\kappa(G)$ using a max-flow/min-cut based approach. The graph is first transformed using vertex splitting so that vertex removal can be modeled as edge removal.

For each pair of vertices (s, t), we treat one as the source and the other as the sink, and repeatedly find an augmenting path from source to sink. For each path, we take the minimum capacity along that path (bottleneck), reduce the capacities accordingly, and update the residual graph. This process is repeated until no more augmenting paths exist.

At this point, the graph is divided into two disjoint sets by the minimum cut, and the edges between these sets represent the minimum number of vertices (or edges) required to disconnect s and t. The minimum value over all pairs gives the vertex connectivity $\kappa(G)$.

Similarly, edge connectivity $\lambda(G)$ is computed without vertex splitting, and the minimum degree $\delta(G)$ is obtained directly. Finally, we verify that $\kappa(G) \leq \lambda(G) \leq \delta(G)$ holds.

Time Complexity: $O(V^2(E \times \text{max\_flow}))$

4.2.5 Whitney 2-Connected Ear Decomposition

Decomposes a 2-connected graph into a cycle and a series of paths (ears) that connect back to already included nodes. This shows the structural composition of the graph.

Use Cases:

- Proving 2-connectivity properties

Time Complexity: O(V + E)

4.3 Shortest Path Algorithms

4.3.1 Dijkstra's Algorithm

Finds the shortest path from a source node to all other nodes in a graph with non-negative edge weights. It greedily picks the closest unvisited node.

Use Cases:

- GPS navigation and route planning
- Network routing protocols
- Social network friend suggestions
- Game pathfinding

How It Works:

1. Set distance to source as 0, all others as infinity
2. Create a priority queue with source node
3. While queue is not empty:

- Extract node with minimum distance
- For each neighbor, check if going through current node gives shorter path
- If yes, update distance and add to queue

Important: Only works with non-negative weights

Time Complexity: O((V + E) log V) with priority queue

4.3.2 Bellman-Ford Algorithm

Finds shortest paths from a source node to all other nodes, but unlike Dijkstra, it can handle negative edge weights. It can also detect negative cycles.

Use Cases:

- Network routing with various costs
- Situations with negative weights (costs vs profits)
- Negative cycle detection

How It Works:

1. Set distance to source as 0, all others as infinity
2. Repeat V-1 times:
   - For each edge (u, v) with weight w:
   - If distance[u] + w ¡ distance[v], update distance[v]
3. Check one more time - if any distance can still be reduced, negative cycle exists

Time Complexity: O(V × E)

4.3.3 Floyd-Warshall Algorithm

Finds shortest paths between all pairs of nodes in the graph. Instead of running Dijkstra from each node, it uses dynamic programming to compute all paths at once.

Use Cases:

- Computing distance matrices
- All-pairs reachability

How It Works:

1. Create a distance matrix, initially with direct edge weights
2. For each intermediate node k:

- For each pair of nodes (i, j):
- Check if path i → k → j is shorter than direct path i → j
- If yes, update distance[i][j]

Time Complexity: $O(V^3)$

4.4 Minimum Spanning Tree Algorithms

4.4.1 Prim's Algorithm

Finds a Minimum Spanning Tree (MST), which is a subset of edges that connects all vertices with the minimum total weight. It builds the tree step by step by always selecting the smallest edge that connects a new vertex.

Use Cases:

- Network design (laying cables, roads)
- Connecting cities with minimum cost

How It Works:

1. Start with any vertex
2. While not all vertices are included:
    - Select the minimum weight edge connecting the tree to a new vertex
    - Add that edge and vertex to the tree

Time Complexity: $O(E \log V)$ using a priority queue

4.4.2 Kruskal's Algorithm

Finds a Minimum Spanning Tree by selecting edges in increasing order of weight and adding them if they do not form a cycle. It uses the Union-Find data structure to detect cycles.

Use Cases:

- Network design problems
- Useful when edges are already sorted
- Sparse graphs (fewer edges)

How It Works:

1. Sort all edges in increasing order of weight

2. Initialize each vertex as a separate set
3. For each edge:
    - If it connects two different sets, add it to MST
    - Merge the sets using Union-Find

Time Complexity: $O(E \log E)$

4.4.3 MST Count Utility

Counts the number of different Minimum Spanning Trees possible in a graph. Multiple MSTs can exist when there are edges with equal weights.

Use Cases:

- Understanding multiple optimal solutions
- Network design flexibility
- Educational purposes

4.5 Network Flow Algorithm

4.5.1 Ford-Fulkerson Algorithm

Finds maximum flow in a flow network - the maximum amount that can flow from source to sink considering edge capacity constraints. It builds a residual graph showing remaining capacities.

Use Cases:

- Network bandwidth optimization
- Circulation problems
- Transportation networks

How It Works:

1. Start with zero flow
2. While there exists a path from source to sink in residual graph:
    - Find augmenting path using BFS/DFS
    - Find minimum capacity along this path
    - Add this flow to total
    - Update residual capacities

Time Complexity: O(E × max_flow).

5 Running the Project(Locally)

From the project root directory:

```
git clone https://github.com/atik-7866/BMP-Graph-Visualizer.git
cd BMP-Graph-Visualizer
cd client
npm install
npm run dev
```

This starts a local development server (usually at `http://localhost:5173`)

6 Challenges Faced

6.1 Visualization Clarity

Problem: Large graphs become cluttered and hard to read.

Solution:

- Limit recommended graph size in documentation
- Use clear color coding

6.2 Directed vs Undirected Graphs

Problem: Different algorithms need different graph types.

Solution:

- Parse input differently based on algorithm
- Show arrow indicators for directed edges
- Validate graph type matches algorithm requirements

6.3 Component Reusability

Problem: Each algorithm needs slightly different visualization.

Solution:

- Create generic GraphVisualization component
- Accept props for customization
- Special components for unique cases (FlowNetworkVisualization)

7 Conclusion

In this project, we developed Graphify, an interactive web application for visualizing graph algorithms. The system allows users to input graphs, run 15 different algorithms, and observe their execution step by step using clear visual representation.

The application successfully combines theory with practice by providing:

- Real-time visualization
- Color-coded states
- Detailed execution logs
- Multiple algorithm categories

By using modern technologies such as React 19, Vite, and Tailwind CSS 4, the application ensures fast performance and smooth user experience.

Graphify serves as a useful learning and teaching tool for graph algorithms. It helps bridge the gap between theoretical knowledge and practical implementation. Users can experiment with different inputs, observe algorithm behavior, and develop deeper understanding of how graph algorithms work. The project demonstrates how visualization can simplify complex concepts and improve understanding.

8 References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
3. React Documentation. *React - A JavaScript library for building user interfaces.* Retrieved from `https://react.dev/`
4. Vite Documentation. *Vite - Next Generation Frontend Tooling.* Retrieved from `https://vitejs.dev/`
5. Tailwind CSS Documentation. *Tailwind CSS - Utility-first CSS framework.* Retrieved from `https://tailwindcss.com/`
6. React Router Documentation. *React Router - Declarative routing for React.* Retrieved from `https://reactrouter.com/`